

Lab Report 5: Scan Line Polygon Fill Algorithm

Name: _____ (Fill in your name)

Lab Program Number: 05

Date: _____ (Fill in your date)

Title

Understanding & Implementing Scan Line Polygon Fill Algorithm

Theory

The Scan Line Polygon Fill Algorithm is a fundamental technique in computer graphics used to fill the interior of polygons efficiently. It works by determining the intersection points of the polygon edges with each horizontal scan line across the display area. Pixels between pairs of sorted intersection points are then filled, taking advantage of the fact that polygons are closed shapes with well-defined boundaries. This method is more efficient than simple boundary-following algorithms as it processes only the relevant scan lines that intersect the polygon.

Key Concepts:

- **Scan Line:** A horizontal line across the screen at a specific y-coordinate
 - **Intersection Points:** Points where the scan line crosses polygon edges
 - **Parity Rule:** Fill between pairs of intersections (1st-2nd, 3rd-4th, etc.)
-

Algorithm Steps

Step 1: Start

Initialize graphics mode and define polygon vertices.

Step 2: Input vertices and define edges

Store vertex coordinates and connect them to form polygon edges.

Step 3: Determine intersection points

For each scan line from top to bottom, calculate where it intersects polygon edges using line equations.

The x-coordinate for an intersection point at scan line y for an edge between (x_1, y_1) and (x_2, y_2) is calculated as:

$$x = x_1 + ((y - y_1) / (y_2 - y_1)) \times (x_2 - x_1)$$

Step 4: Sort intersection points

Arrange intersection x-coordinates in ascending order for each scan line.

Step 5: Fill between intersection pairs

For each pair of sorted intersection points (1st & 2nd, 3rd & 4th, etc.), fill all pixels horizontally between them.

Step 6: Repeat for all scan lines

Continue process until entire polygon area is covered.

Step 7: Stop

Close graphics mode when filling is complete.

Code

```
c
```

```

#include <graphics.h>
#include <stdio.h>

void scanlinePolygonFill(int polygon[][2], int n) {
    int y, x, i, j;
    int x_intersect[20]; // Array to store intersection points for current scan line
    int gd = DETECT, gm;
    initgraph(&gd, &gm, ""); // Initialize graphics mode

    // Process each scan line from top to bottom of screen
    for (y = 0; y < getmaxy(); y++) {
        int k = 0; // Counter for number of intersections on current scan line

        // Find intersections with all polygon edges
        for (i = 0; i < n; i++) {
            j = (i + 1) % n; // Next vertex (wraps around to first)

            // Check if scan line intersects this edge
            if ((polygon[i][1] <= y && polygon[j][1] > y) ||
                (polygon[j][1] <= y && polygon[i][1] > y)) {

                // Calculate x-coordinate of intersection point
                x_intersect[k++] = (y - polygon[i][1]) *
                    (polygon[j][0] - polygon[i][0]) /
                    (polygon[j][1] - polygon[i][1]) +
                    polygon[i][0];
            }
        }

        // Sort intersection points in ascending order (using bubble sort)
        for (i = 0; i < k - 1; i++) {
            for (j = i + 1; j < k; j++) {
                if (x_intersect[i] > x_intersect[j]) {
                    int temp = x_intersect[i];
                    x_intersect[i] = x_intersect[j];
                    x_intersect[j] = temp;
                }
            }
        }

        // Fill pixels between pairs of intersection points
        for (i = 0; i < k; i += 2) {
            for (x = x_intersect[i]; x <= x_intersect[i + 1]; x++) {

```

```

        putpixel(x, y, WHITE);
    }
}

getch(); // Wait for user key press
closegraph(); // Close graphics mode
}

int main() {
    int numVertices, i;
    int vertices[10][2]; // Array to store up to 10 vertices (x, y)

    printf("Enter number of vertices for polygon (maximum 10): ");
    scanf("%d", &numVertices);

    printf("Enter the coordinates (x y) for each vertex:\n");
    for (i = 0; i < numVertices; i++) {
        printf("Vertex %d: ", i + 1);
        scanf("%d %d", &vertices[i][0], &vertices[i][1]);
    }

    // Call the scanline fill function
    scanlinePolygonFill(vertices, numVertices);

    return 0;
}

```

Output

(Describe or attach your output here)

Example:

The program displays a graphics window showing a filled polygon based on the vertices entered. For a triangle with vertices (100, 100), (200, 100), (150, 200), the output would be a filled triangle. The algorithm correctly fills the interior by processing each horizontal scan line and filling between intersection pairs.

Conclusion

This lab successfully implemented the Scan Line Polygon Fill Algorithm. The program demonstrates the

algorithm's effectiveness in filling polygon interiors by systematically processing horizontal scan lines, calculating edge intersections, and filling between sorted intersection pairs.

Key learnings:

- The algorithm's efficiency comes from processing only relevant scan lines
- Integer arithmetic is used for intersection calculations, improving performance
- The parity rule (filling between pairs) ensures correct filling for complex polygons
- Sorting intersections is crucial for correct fill behavior

This exercise provided practical experience with fundamental raster graphics techniques and polygon filling methods commonly used in computer graphics applications, game engines, and rendering systems.

Instructor's Signature: _____

Date: _____

Marks: _____ / _____